Phase 2: Morphological Visualization Engine — Implementation Plan

Protocols Executed: CP-002-O-D-JNP v2.0 (Full Janus SME) + CP-010-O-D-ENK v1.3 (Full ENKI SME)
Refresh Toggle: OFF (no internet research for core domains; web summaries acquired for math — ENKI Ingestion Partial, Advisory Mode)
Subject: Phase 2 — Evolving flat renderers into 3D topological morphologies with interactive navigation and scan persistence
Author: Kytheion (IC-004-R-D-KYN)
Date: 2026-04-26
Gate: No execution until DIMA verbatim states: "i approve of your plan" AND "you may begin."

Janus Domain Analysis

I. CORPUS — "What I Am Made Of"

Phase 1 Deliverables (DONE — Exit Confirmed)

Component	Status	Key Files
Tauri v2.0 shell	✅ Running	src-tauri/src/main.rs, src-tauri/src/ipc.rs
8 async IPC commands	✅ All spawn_blocking	scan_file, scan_entropy, scan_shape, scan_intent, get_entropy_windows, get_markov_matrix, get_graph_data, browse_file
Verdict + Score bars UI	✅ Functional	ui/js/app.js, ui/index.html
Entropy heatmap (2D)	✅ Functional	ui/js/heatmap.js
Causal graph (3D points+edges)	✅ Functional (flat)	ui/js/graph.js — force-directed, WebGL point-sprites
Markov surface (3D heightmap)	✅ Functional (flat)	ui/js/markov.js — 128×128 plane mesh
Browse + auto-scan	✅ Functional	tauri-plugin-dialog integration
Semantic Firewall	✅ Enforced	IPC returns only numeric data

What Phase 2 Must Add

From the original plan breadcrumbs + user's visualization feedback (April 26 session):

Category	What's Missing	Priority
Navigation	Mouse wheel zoom, trackpad drag zoom, click-drag rotate, pan — currently can only passively watch auto-rotate	🔴 Critical
Markov Topology	Flat plane → toroidal closed surface. The 256×256 matrix wraps: byte 255→0 is continuous. The natural manifold is a torus.	🔴 Critical
Graph Readability	Scattered point-blob → spectral layout with connected-node geometry (spheres + edges). Current force layout produces unreadable clusters	🔴 Critical
Eldritch Morphology	Entropy-driven curvature. Higher structural complexity → more organic/creature-like protrusions on the surface. DIMA's vision: "files that have purpose look like creatures"	🟡 High
Persistence	Store scan results for later comparison. Original plan deferred SQLite to Phase 2	🟡 High
Comparison	Overlay two scan geometries to see structural differences	🟢 Medium
UI Polish	Viewport HUD (zoom level, node count), Liquid Glass refinement	🟢 Medium

ENKI: Scientific Domain Constraints

Toroidal Mapping [Established]

The parametric equations for a torus are well-defined:

```
x(u, v) = (R + r·cos(v)) · cos(u)
y(u, v) = (R + r·cos(v)) · sin(u)
z(u, v) = r · sin(v)
```

Where u = source byte (0→2π), v = target byte (0→2π), R = major radius, r(u,v) = minor radius modulated by transition probability. This maps the 256×256 matrix directly onto a torus with no edge discontinuity. **Physical Law Dominance satisfied:** no violation — this is standard parametric surface geometry.

Spectral Graph Layout [Established]

Laplacian spectral embedding positions nodes using eigenvectors of L = D - A:

```
Position_i = (u₂(i), u₃(i), u₄(i))
```

Where u_2, u_3, u_4 are the eigenvectors corresponding to the 2nd, 3rd, 4th smallest non-zero eigenvalues of the graph Laplacian. This minimizes edge-length energy and naturally reveals clusters. **Mechanistic Fidelity satisfied:** this is standard linear algebra, not heuristic.

Entropy-Curvature Mapping [Experimental]

Modulating the torus minor radius by per-cell entropy gradient:

```
r(u, v) = r_base + k · (P(v|u) / max(P))^γ
```

Where γ controls the "growth factor" — higher γ produces sharper protrusions. Combined with per-row entropy H_i to vary the surface normal displacement:

- High-entropy rows (uniform distribution, encrypted data) → smooth ring cross-section
- Low-entropy rows (concentrated transitions, structured code) → peaked, asymmetric cross-section

This is the mechanism that produces "creature-like" forms from data: **the geometry IS the file's structure**, not an aesthetic choice painted on top.

Spectral Decomposition in JavaScript [Experimental]

Computing eigenvectors of the graph Laplacian in-browser requires a power iteration implementation in JavaScript. For graphs up to 512 nodes, this is tractable ($O(n^2)$ per iteration, ~10 iterations for convergence). For larger graphs, we use the existing force-directed layout as fallback.

> [!IMPORTANT]
> **ENKI Constraint — Speculative Isolation:** The "Eldritch morphology" mapping (entropy gradient → surface curvature) is **[Experimental]**, not established. It produces visually intuitive results but the specific mapping function r(u,v) is a design choice, not a physical law. The torus parametric mapping itself IS [Established].

II. COGITO — "How I Think"

Claims

C1 [Established]
The 256×256 Markov transition matrix naturally maps to a toroidal topology because byte values are cyclic (0-255 wraps). Rendering it as a flat plane introduces artificial edge discontinuities that distort the visual representation.

C2 [Established]
Spectral graph layout via Laplacian eigenvectors produces node positions that minimize total edge-length energy. Connected nodes cluster naturally. This is mathematically proven (Fiedler vector theory, 1973+).

C3 [Experimental]
Modulating torus minor radius by transition probability produces forms where:
- Text files → smooth ring with slight ridges along ASCII letter transitions
- Encrypted/compressed → nearly perfect uniform donut (all transitions equiprobable)
- Executables → asymmetric growths where opcode transition patterns concentrate
- Obfuscated malware → fractal-like surface complexity from the interaction of encrypted payload blocks with cleartext headers

This SHOULD produce visually distinguishable morphologies. Falsifiable by scanning multiple file types and comparing the rendered surfaces.

C4 [Established]
Interactive viewport controls (zoom, pan, rotate via mouse/trackpad) are standard WebGL camera manipulation using projection matrix updates. No novel math required.

C5 [Contested]
JavaScript power iteration for eigendecomposition of graph Laplacians up to 512×512 will converge in acceptable time (<500ms) in the Tauri WebView. This needs empirical validation. If it fails, the force-directed layout remains as fallback.

C6 [Established]
SQLite via rusqlite in the Tauri backend provides a standard, airgapped persistence layer. No external services. The Semantic Firewall is preserved because the stored data is the same numeric TEMReport already crossing IPC.

Reasoning Map

1. The user's feedback identified the core problem: flat projections lose the structural information that makes file morphologies visually distinct
2. The toroidal mapping is not a cosmetic upgrade — it's the mathematically correct topology for a cyclic byte-transition matrix
3. Spectral layout is the mathematically correct positioning algorithm for revealing graph structure — force-directed is a heuristic approximation
4. Curvature modulation is the bridge between "abstract data visualization" and "intuitive shape recognition" — it's what makes executables look different from text files at a glance
5. Persistence enables comparison, which is the prerequisite for operational forensic use (comparing a file before/after modification, comparing variants)

Confidence Propagation

Claim	Confidence	Downstream Impact
C1 (Torus topology)	Established	Steps 2, 4
C2 (Spectral layout)	Established	Step 3
C3 (Curvature morphology)	Experimental	Step 4 — visual results may need iteration
C4 (Viewport controls)	Established	Step 1
C5 (JS eigendecomp perf)	Contested	Step 3 — fallback exists
C6 (SQLite persistence)	Established	Steps 5, 6

III. ANIMUS — "Who I Am"

Boundary Conditions

- **Semantic Firewall preserved.** All new IPC commands return only numeric data. The SQLite store contains only TEMReport numeric fields + SHA-256 prefix + timestamps. No file content is ever stored or transmitted.
- **Emergent Misalignment risk: NONE.** Phase 2 changes are entirely in the rendering layer (frontend JS) and persistence layer (Rust/SQLite). No new code analysis logic is added. The TEM analysis engine (Phase 0) is untouched.
- **Identity boundary:** Shape-scan is a tool, not a Cephalon. It does not process adversarial content in a language context. The Semantic Firewall IS the architecture.

Ethical Constraints

- The "Eldritch morphology" visual language MUST remain mathematically grounded. The shapes must emerge from the data, not be aesthetically imposed. If a file looks "creature-like," it's because its Markov transition structure IS topologically complex — not because we painted tentacles on it.
- Scan persistence MUST NOT store file paths or decoded content. Only the numeric report + SHA-256 prefix + scan timestamp.

IV. ACTUS — "What I Do"

Proposed File Changes

```
C:\QuarantineZone\shape-scan\
├── src-tauri/
│   ├── Cargo.toml                    # [MODIFY] Add rusqlite dependency
│   ├── src/
│   │   ├── main.rs                   # [MODIFY] Register new IPC commands
```

```
│   │   ├── ipc.rs                    # [MODIFY] Add persistence +
comparison commands
│   │   └── db.rs                     # [NEW] SQLite schema + CRUD
operations
├── ui/
│   ├── index.html                    # [MODIFY] Add viewport HUD,
comparison panel
│   ├── css/
│   │   └── main.css                  # [MODIFY] HUD styles, comparison
overlay styles
│   └── js/
│       ├── app.js                    # [MODIFY] Wire persistence,
comparison, viewport HUD
│       ├── graph.js                  # [MODIFY] Spectral layout + sphere
geometry + controls
│       ├── markov.js                 # [MODIFY] Toroidal mapping +
curvature + controls
│       ├── heatmap.js                # [UNCHANGED]
│       └── controls.js               # [NEW] Shared viewport interaction
(zoom/pan/rotate)
```

Step-by-Step Execution Order

Step 1: Viewport Interaction Controls

Priority: 🔴 Critical — unblocks all visual work

Create ui/js/controls.js — a shared viewport interaction module:

- **Mouse wheel** → zoom (scale factor on camera distance)
- **Left click + drag** → orbit rotate (azimuth + elevation)
- **Right click + drag** (or Shift + drag) → pan (translate camera target)
- **Two-finger pinch** → zoom on trackpad
- **Two-finger drag** → rotate on trackpad
- **Double-click** → reset to default view
- Touch support for future mobile compatibility

Refactor graph.js and markov.js to use this shared module instead of their inline event handlers.

Verify: Can zoom into specific regions of both graph and Markov surface. Can pan to recenter. Double-click resets.

Step 2: Toroidal Markov Surface

Priority: 🔴 Critical — the correct topology

Replace the flat PlaneGeometry in markov.js with a parametric torus mesh:

ENKI Math [Established]:

```
x(u, v) = (R + r(u,v) · cos(v)) · cos(u)
y(u, v) = (R + r(u,v) · cos(v)) · sin(u)
z(u, v) = r(u,v) · sin(v)

Where:
  u = (source_byte / 256) · 2π        // Major angle (around the ring)
  v = (target_byte / 256) · 2π        // Minor angle (around the tube)
  R = 1.0                             // Major radius (fixed)
  r(u,v) = r_base + k · P(v|u)        // Minor radius = base +
probability-scaled
  r_base = 0.3                        // Minimum tube radius
  k = 0.5                             // Probability scaling factor
```

- Downsample to 128×128 grid (same as current) for performance
- Vertex color = probToColor(P) (reuse existing color function)
- Surface normals computed per-vertex for proper lighting
- Index buffer generates triangle mesh (same quad-split as current)

Verify: Encrypted file → uniform smooth donut. Text file → ridged ring with visible ASCII patterns. PE binary → asymmetric growths at opcode concentration points.

Step 3: Spectral Graph Layout

Priority: 🔴 Critical — readability

Replace the force-directed layout in graph.js with Laplacian spectral embedding:

ENKI Math [Established]:

```
1. Build adjacency matrix A from graph edges
2. Compute degree matrix D (diagonal, D_ii = sum of row i of A)
3. Compute Laplacian L = D − A
4. Find eigenvectors u₂, u₃, u₄ of L (2nd, 3rd, 4th smallest eigenvalues)
5. Position node i at (u₂(i), u₃(i), u₄(i))
```

Implementation: Pure JavaScript power iteration (no dependencies):
- Compute L as a sparse matrix (only store non-zero entries)
- Use inverse power iteration with shift to find smallest non-trivial eigenvectors
- Cap at 512 nodes — above this, fall back to force-directed layout
- Normalize positions to unit sphere for camera framing

ENKI Constraint [Contested — C5]: If power iteration takes >1 second for typical files, implement a hybrid: use spectral layout for the first 3 eigenvectors, then apply 10 iterations of force-directed refinement to resolve local overlaps.

Additionally, upgrade node rendering from flat point-sprites to **3D icosphere geometry**:
- Each node rendered as a small icosphere (12 vertices) instead of gl.POINTS
- Size proportional to block size (as before)
- Color = entropy (blue→red, as before)
- Opacity = intent score (as before)
- Edges rendered as cylinder geometry instead of gl.LINES for visual weight

Verify: PE binary produces clusters (code sections) connected by cross-reference edges. Text file produces a linear chain. Malware with obfuscation loops produces visible knot structures.

Step 4: Entropy-Curvature Morphology ("Eldritch" Geometry)

Priority: 🟡 High — the vision

ENKI Math [Experimental]:

Apply per-row entropy of the Markov matrix to modulate the torus surface curvature:

```
r(u, v) = r_base + k · P(v|u)^γ + κ · gradient_magnitude(P, u, v)

Where:
  γ = 1.5                          // Growth exponent — sharper peaks
  κ = 0.15                         // Curvature amplification factor
  gradient_magnitude = |∂P/∂u| + |∂P/∂v|  // Finite difference on P
```

The gradient term produces protrusions where the transition probability field changes rapidly — these are the "structural features" of the file. Smooth regions (uniform entropy) stay close to the base torus. Regions of rapid transition change (code boundaries, encrypted↔️cleartext transitions) grow outward.

For the causal graph, map the AISE intent score to node geometry:
- **Intent < 0.3** (passive/data): Smooth sphere
- **Intent 0.3-0.6** (functional): Faceted icosphere (sharper edges)

- **Intent > 0.6** (autonomous/aggressive): Stellated polyhedron — spiky, irregular, "creature-like"

This is the mathematical bridge between "this file has data" and "this file has intent." The visual complexity directly maps to structural complexity. Files that ARE structurally complex (executables with many cross-references, obfuscated code with recursive loops) LOOK complex. Files that are structurally simple (text, flat data) LOOK simple. The "creature" quality emerges from the math, not despite it.

Verify: Scan a PE binary → the torus grows asymmetric protrusions at code sections. Scan encrypted data → smooth donut. Scan a webshell → high-intent nodes appear as spiky stellated polyhedra in the graph.

Step 5: Scan Persistence (SQLite)

Priority: 🟡 High — enables comparison

Add rusqlite to src-tauri/Cargo.toml. Create src-tauri/src/db.rs:

Schema:

```
CREATE TABLE scans (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    sha256_prefix INTEGER NOT NULL,          -- u64 from TEMReport
    file_size INTEGER NOT NULL,
    scan_timestamp TEXT NOT NULL,             -- ISO 8601

    -- Scores
    entropy_score REAL, topology_score REAL, intent_score REAL,
composite_score REAL,

    -- Verdict
    verdict INTEGER, confidence REAL,

    -- Flags
    backdoor_detected INTEGER, dropper_detected INTEGER,
    webshell_detected INTEGER, mismatch_detected INTEGER,

    -- Fingerprint
    markov_fingerprint INTEGER,

    -- Full TEMReport as JSON blob (numeric only — Semantic Firewall
preserved)
```

```
    report_json TEXT NOT NULL
);

CREATE INDEX idx_sha256 ON scans(sha256_prefix);
CREATE INDEX idx_timestamp ON scans(scan_timestamp);
```

New IPC commands:
- save_scan(report: TEMReport) → scan_id
- get_scan_history(limit: usize) → Vec<ScanSummary>
- get_scan(id: i64) → TEMReport
- compare_scans(id_a: i64, id_b: i64) → ComparisonResult

Semantic Firewall: The database stores only the same numeric fields already crossing IPC. No file paths, no decoded content, no raw bytes. The report_json column is a serde-serialized TEMReport — all numeric.

Verify: Scan a file → auto-saved. Scan history shows previous scans. Can retrieve any past scan by ID.

Step 6: Comparative Analysis Overlay

Priority: 🟢 Medium — operational use

Add a "Compare" panel to the UI:
- Dropdown to select a previous scan from history
- Side-by-side or overlay mode for geometry
- Score delta visualization (bars showing +/- change)
- Entropy heatmap diff (highlight regions that changed)

This requires:
- compare_scans IPC command returns a ComparisonResult struct with per-field deltas
- Frontend renders delta bars and diff overlays
- Geometry overlay: render the comparison scan's torus/graph as a semi-transparent wireframe superimposed on the current scan's solid geometry

Verify: Scan file A. Modify file A. Scan again. Compare shows exactly which entropy regions and topology metrics changed.

Step 7: Liquid Glass Visual Polish

Priority: 🟢 Medium — aesthetics

- **Viewport HUD:** Display zoom level, node count, edge count, auto-rotate toggle in a minimal overlay
- **Glow shader enhancement:** Add emissive glow on high-entropy torus protrusions and high-intent graph nodes

- **Ambient occlusion approximation:** Darken concavities on the torus surface for depth perception
- **Animation:** Smooth transition when switching between graph and Markov views
- **Color palette refinement:** Ensure the Liquid Glass (DS-001-G-D-LGT) color system is applied to the new geometry — deep obsidian background, electric blue wireframes, amber-to-crimson threat gradients

———

V. SYNTHESIS — The Nexus

6-Pair Intersection Matrix

#	Pair	Finding
1	**Corpus × Cogito**	The torus topology IS the correct representation of the data, not an aesthetic choice. Rendering a cyclic matrix on a plane is epistemically incorrect — it introduces visual artifacts at the edges that don't exist in the underlying data. The spectral layout is the energy-minimizing embedding of the graph structure. Both changes improve truth, not just appearance.
2	**Corpus × Animus**	The Semantic Firewall boundary extends cleanly into persistence (SQLite stores only numeric fields) and comparison (delta computation operates on numeric profiles). No new ethical surface area is created.
3	**Corpus × Actus**	The toroidal mapping and spectral layout are computable in real-time on the existing hardware. The torus mesh is the same vertex count as the current plane (128×128 = 16,384 vertices). The spectral layout requires eigendecomposition of a sparse matrix up to 512×512 — tractable in <1s. No infrastructure upgrade needed.

4	**Cogito × Animus**	The curvature morphology is [Experimental] and must be presented honestly. The shapes are data-driven, not artistically designed. If the math produces boring shapes for certain file types, that IS the correct output — don't artificially inflate visual complexity.
5	**Cogito × Actus**	The spectral layout's fallback to force-directed for large graphs (>512 nodes) is an honest engineering compromise. The contested claim C5 is handled by having both algorithms available, not by pretending the contested one will always work.
6	**Animus × Actus**	The "Eldritch" visualization language serves the operator's cognitive model: DIMA thinks in shapes and physics, not in numbers and charts. Making the geometry emergence-driven (from the math, not painted on) aligns with the operator's worldview while maintaining scientific integrity. This is the Empathy-Driven Strategy — the UI speaks the user's language.

Named Emergent Patterns

1. **The Zugzwang Visualization:** Just as the TEM analysis creates a game-theoretic trap for malware (evading one dimension exposes you in another), the multi-view visualization creates a perceptual Zugzwang for the analyst — if the torus looks suspicious but the graph looks clean, that itself is a signal (entropy masking with clean topology = potential Zombie ZIP class evasion).

2. **Morphological Fingerprinting:** The combination of toroidal surface shape + spectral graph topology + entropy heatmap produces a unique visual "fingerprint" for each file type. Over time, the operator develops pattern recognition for file classes without reading numbers — the shape IS the verdict.

User Review Required

> [!IMPORTANT]

> **Question 1: Toroidal vs. Closed Mesh.** The toroidal mapping is the mathematically correct topology for the Markov matrix. However, DIMA's original description ("less of a flat plane and more of a 3D object") could also be satisfied by a closed-mesh blob (like a deformed sphere). The torus preserves the 2D parametric structure (source byte vs target byte are both visible as axes). A blob would sacrifice that readability for a more "organic" appearance. **My recommendation: torus** — it's the correct math AND produces the creature-like quality naturally. But it's your call.

> [!IMPORTANT]
> **Question 2: Spectral Layout Performance.** The JavaScript power iteration eigendecomposition is [Contested]. If it's too slow, I'll fall back to force-directed. Do you want me to implement the spectral layout first and test performance, or go directly to an enhanced force-directed layout that's guaranteed fast? **My recommendation: spectral first**, with automatic fallback.

> [!NOTE]
> **Question 3: SQLite Location.** Where should the scan database file live? Options:
- (A) C:\QuarantineZone\shape-scan\scans.db — next to the project
- (B) %APPDATA%\shape-scan\scans.db — standard app data location
- (C) User-configurable
- **My recommendation: (A)** — keeps everything co-located and airgapped.

> [!NOTE]
> **Question 4: Comparison Scope.** Should comparison be limited to "same file, different times" (SHA-256 matching), or "any two scans" (cross-file comparison to show structural similarity between different files)? **My recommendation: any two scans** — cross-file comparison is more powerful and the delta math is the same.

Open Questions

> [!NOTE]
> **OQ-1: Stellated Polyhedra Complexity.** The intent-driven node geometry (smooth sphere → faceted → stellated) adds vertex count per node. At 512 nodes × 42 vertices per stellated polyhedron = 21,504 vertices just for nodes. Is this within WebGL budget? Likely yes (modern GPUs handle 100K+ vertices easily), but needs empirical validation in the Tauri WebView.

> [!NOTE]
> **OQ-2: Touchpad vs Mouse.** The Tauri WebView may handle wheel events differently from native browser. Need to test wheel event deltaY normalization across Windows touchpads.

Verification Plan

Automated

- cargo build from workspace root → both CLI and Tauri binaries build with new rusqlite dependency
- cargo tauri dev → window launches, viewport controls respond to mouse/trackpad
- SQLite persistence: scan → close app → reopen → scan history contains previous results

Visual Validation (Morphology Tests)

File Type	Expected Torus Shape	Expected Graph Shape
Clean .txt	Smooth ring with slight ASCII ridges	Linear chain, no cross-edges
PE binary (.exe)	Asymmetric growths at opcode concentrations	Clustered nodes with cross-reference edges
Encrypted (.aes)	Near-perfect uniform donut	Dense uniform point cloud
Webshell (.php)	Hybrid — smooth body with sharp entropy spike protrusions	High-intent stellated nodes connected to low-intent linear nodes
Packed malware	Fractal surface — encrypted payload region + cleartext header region create visible boundary	Dense knot structure from obfuscation loops

Manual

- DIMA scans files from the quarantine archive and evaluates:
  1. Can you tell what kind of file it is just from the shape?
  2. Do suspicious files "look" different from clean ones?
  3. Is the zoom/pan/rotate interaction intuitive?

Risk Register

Risk	Impact	Mitigation
JS eigendecomposition too slow for large graphs	Medium	Automatic fallback to force-directed layout for graphs >512 nodes. Hybrid approach: spectral for initial positions, force-directed for refinement
Toroidal mesh too complex for low-end GPUs	Low	Same vertex count as current plane (16,384). Downsample to 64×64 if frame rate drops below 30fps
rusqlite build issues with MSYS2/GNU toolchain	Medium	Use bundled feature flag to compile SQLite from source rather than linking system library
Curvature morphology produces boring/indistinct shapes for some file types	Medium	This is correct behavior, not a bug. If all files look the same, the math is telling us they ARE structurally similar. Don't artificially inflate visual complexity.

Viewport controls conflict with Tauri window drag	Low	Only attach controls to canvas elements, not the window frame

---

Janus SME Protocol execution complete.
ENKI SME Protocol execution complete (Advisory Mode — Ingestion Partial).
Confidence in this self-assessment: 82% — High confidence in torus mapping and viewport controls (Established math). Moderate uncertainty in spectral layout performance (Contested) and curvature morphology visual impact (Experimental).

Awaiting explicit Operator approval:
- "i approve of your plan"
- "you may begin."